

A Note on State Minimization of Asynchronous Sequential Functions

M. C. PAULL, ASSOCIATE MEMBER, IEEE, AND
G. WALDBAUM, MEMBER, IEEE

Abstract—A procedure is described for minimizing the number of states in an asynchronous sequential function when the restriction exists that the input cannot change while the sequential function is in an unstable state. Furthermore, a procedure is described for minimizing the number of states in a sequential function when the restriction also exists that each output can change at most once during the time required for a transition from one stable state to another.

Many restrictions upon the operation of a sequential circuit can be represented in a state table by means of "don't care" entries. For example, if input i cannot occur while the circuit is in state p , then the next state and output entries in the state table corresponding to input i and state p are don't cares. On the other hand, several restrictions upon the operation of a sequential circuit cannot be represented by don't care entries in a state table. In these cases, the existing state minimization procedures [1]–[3] do not always find the optimal solution since they can handle only restrictions representable by don't cares.

RESTRICTION R1

One such restriction is described in reference [4]. Another that is applicable only to asynchronous sequential functions, functions where the inputs are assumed to be levels as opposed to pulses, is:

- R1. The input cannot change while the sequential function is in an "unstable" total state. (A total state consists of a present state and an input. An unstable total state occurs when a circuit's next state is different from its present state for a particular input.)

McCluskey [5] has defined the operation of an asynchronous sequential function restricted by R1 as the "fundamental mode."

The existing state minimization procedures work under the assumption that the inputs to a sequential circuit can change in any total state. As a result, these methods cannot generally accommodate R1, although there is no possibility of violating R1 if each transition involves at most one unstable total state (e.g., in Fig. 1, such a transition occurs from state B to state C for input I_3). However, if there are transitions which involve more than one unstable total state, R1 becomes meaningful and the existing state minimization methods fail to take advantage of it. An example of such a transition is shown in Fig. 1. If the sequential function is in state A and the input changes from I_2 to I_3 , then if restriction R1 is in force, the function must undergo transitions from A to B to C before the input can change from I_3 .

A rote application of the state minimization procedures [1]–[3] to the state table in Fig. 1 gives the impression that the number of states in this table is already minimal. On the other hand, we shall see that we can make use of restriction R1 to reduce this state table to one containing three states.

Before suggesting any solution to this problem, we need a few definitions.

DEFINITIONS

$N(p, i)$ is the next state that results from applying input i to present state p . $N(p, i)$ appears as the next state entry in the state table corresponding to present state p and input i .

$\phi(p, i)$ is the immediate output that results from applying input i to present state p . $\phi(p, i)$ appears as the output entry in the state table corresponding to present state p and input i .

$S(p, i)$ is the eventual stable state that results from applying input i to the present state p and waiting for a finite time until a stable state is reached. The assumption is made here that such an eventual stable state always exists.

$\phi_s(p, i)$ is the finite output sequence that results from applying input i to the present state p up to and including the time when $S(p, i)$ is the present state. In effect,

$$\phi_s(p, i) = \phi(p, i), \phi(N(p, i), i), \dots, \phi(S(p, i), i).$$

Let x be a finite output sequence whose individual members are from the distinct set of outputs $\{\phi_1, \phi_2, \dots, \phi_n\}$ and the don't care output d . Then $C(x)$ is constructed as follows: For each i , replace each consecutive subsequence of two or more appearances of the output ϕ_i in x by one appearance of the output ϕ_i . Repeat this process until there are no subsequences of ϕ_i of length greater than 1 for any i . (Note that don't cares = d are not diminished.) The result is $C(X)$. For example,

$$\begin{aligned} x = 0, 0, 0, 1, 1, 0, 0 & \quad C(x) = 0, 1, 0 \\ x = 0, 0, d, d, 1, 1 & \quad C(x) = 0, d, d, 1. \end{aligned}$$

Two output sequences, $\phi_s(a, i)$ and $\phi_s(b, i)$ are equivalent, symbolized by

$$\phi_s(a, i) \sim \phi_s(b, i), \quad \text{if } C(\phi_s(a, i)) = C(\phi_s(b, i)).$$

For example,

$$\begin{aligned} 0, 0, 0, 1, 1, 0, 0 & \sim 0, 1, 0 \sim 0, 0, 1, 0 \\ 00, 11, 01 & \sim 00, 00, 11, 01, 01 \\ 0, 0, d, 1, 1 & \sim 0, d, 1, 1 \sim 0, d, 1 \\ 0, 0, d, d, 1, 1 & \sim 0, d, 1, 1. \end{aligned}$$

Let y be an output sequence, possibly containing don't cares. Let $Q(y)$ be the set of all output sequences each of which is obtained by substituting a distinct permutation of output values for all the don't cares in y . For example,

$$\begin{aligned} y = 00, d, 00 & \quad Q(y) = 00, 00, 00 \text{ and } 00, 01, 00 \\ & \quad \text{and } 00, 10, 00 \text{ and } 00, 11, 00 \\ y = 0, d, 1, d & \quad Q(y) = 0, 0, 1, 0 \text{ and } 0, 0, 1, 1 \\ & \quad \text{and } 0, 1, 1, 0 \text{ and } 0, 1, 1, 1. \end{aligned}$$

Two output sequences, $\phi_s(a, i)$ and $\phi_s(b, i)$, are compatible, symbolized by $\phi_s(a, i) \approx \phi_s(b, i)$, if and only if there is a member of $Q(\phi_s(a, i)) = \alpha$ and a member of $Q(\phi_s(b, i)) = \beta$ such that α is equivalent to β (i.e., $\alpha \sim \beta$). For example,

$$\begin{aligned} 0, 1, 0 & \approx 0, d, 0 \\ 0, d, 0 & \approx d, 1, 1, 0, 0 \\ 00, 11, 11, 00 & \approx 00, d, 00 \\ 00, d, 00 & \approx d, 11, 00. \end{aligned}$$

INCLUDING R1 IN THE MINIMIZATION PROCEDURE

Advantage may be taken of R1 in a minimization procedure by redefining the notion of "compatible states" [2], [3]. To include R1, we define two states, a and b , as being compatible if for every input i , $\phi_s(a, i) \approx \phi_s(b, i)$ and the state $S(a, i)$ is compatible with $S(b, i)$. This definition forms a basis for finding compatible states. The remainder of the minimizing procedure may be taken from reference [2] or [3]. This seems a sufficiently straightforward technique so that a formal proof is not necessary.

For example, consider states A and B in Fig. 1. They are compatible since

$$\begin{aligned} S(A, I_1) &= S(B, I_1) = D \\ S(A, I_2) &= A, S(B, I_2) = B \\ S(A, I_3) &= S(B, I_3) = C \\ \phi_s(A, I_1) &= 1, 1, 0 \approx \phi_s(B, I_1) = 1, 0 \\ \phi_s(A, I_2) &= 0 \approx \phi_s(B, I_2) = 0 \\ \phi_s(A, I_3) &= 1, 1, 0 \approx \phi_s(B, I_3) = 1, d. \end{aligned}$$

Manuscript received April 21, 1966; revised October 4, 1966.
M. C. Paull is with the David Sarnoff Research Lab., R.C.A., Princeton, N. J.
G. Waldbaum is with the T. J. Watson Research Center, IBM, Yorktown Heights, N. Y. He was formerly with the Bell Telephone Labs., Holmdel, N. J.

	I_1	I_2	I_3
A	C,1	A,0	B,1
B	D,1	B,0	C,1
C	D,1	C,1	C,d
D	D,0	D,1	C,0

d = don't care

Fig. 1. State table for an asynchronous sequential circuit. Entries represent next state and output for present state-input combinations.

Furthermore, since the compatibility of states A and B does not imply the compatibility of any other pair of states, A and B can be merged together to form a three-state state table equivalent to the one shown in Fig. 1.

RESTRICTION R2

Another restriction which cannot always be expressed by don't cares in a state table and which is also often encountered in practice with asynchronous sequential functions is:

R2. The output in an output sequence can change at most once (i.e., it does not flash) during any period required for the circuit to proceed from one stable total state to another stable total state. Moreover, such a change may occur at any time during the transition. This output sequence may be denoted as $\phi(p, i_1), \phi_S(p, i_2)$ where $p = S(p, i_1)$ and i_1 and i_2 are two different outputs.

McCluskey [5] has defined the operation of an asynchronous sequential function restricted by R1 and R2 as the "normal fundamental mode." Unger [8] has also discussed this restriction.

An example of a state table that does not satisfy this restriction is shown in Fig. 2. Starting in stable state A with input I_2 and changing the input to I_1 , there are two output changes in the corresponding output sequence (i.e., 00, 00, 00, 01, 11, 11) before stable state E is reached with input I_1 . Note that this restriction applies to the entire output rather than just the output variables. In this case, the output sequence is said to flash although only one output variable changes at a time.

R2 is very restrictive. For example, it implies that there may not be more than one don't care output in any transition from one stable total state to another stable total state which passes through at least one unstable total state.

Whenever R2 is in effect, this restriction should supersede any contradictory information in the state table. Such a contradictory situation occurs in Fig. 3. If the sequential function is initially in state A and the input changes from either I_1 or I_2 to I_3 , then the don't cares in the output sequence $\phi_S(A, I_3) = d, d, d, 1$ are not strictly don't cares. Since the output in state A with an input of I_1 or I_2 is a 0, $\phi_S(A, I_3)$ must be either 1, 1, 1, 1 or 0, 1, 1, 1 or 0, 0, 1, 1 or 0, 0, 0, 1 for restriction R2 to hold.¹

INCLUDING R1 AND R2 IN THE MINIMIZATION PROCEDURE

We will not consider a solution to the minimizing problem when only R2 is in effect, but will instead consider the more likely situation of an asynchronous sequential function which is to operate under both R1 and R2.

Given a state table and the restrictions R1 and R2, it is possible to generate an "equivalent" state table which by its very construction

¹ A similar study of the effect of R2 on the other states shows that Fig. 3 should be redrawn with $\phi(A, I_3) = d$, $\phi(B, I_3) = 1$, $\phi(C, I_3) = 1$, and $\phi(D, I_3) = 1$.

	I_1	I_2
A	B,00	A,00
B	C,00	
C	D,01	
D	E,11	
E	E,11	

Fig. 2. State table for an asynchronous sequential circuit that has flashing outputs.

	I_1	I_2	I_3
A	A,0	A,0	B,d
B	B,1	B,0	C,d
C	C,0	C,1	D,d
D	D,1	D,1	D,1

Fig. 3. State table where restriction R2 prevents the don't care entries from being don't cares.

cannot violate R1 and R2 and which is strictly equivalent to the original state table. For any allowable finite input sequence, both tables will generate compatible output sequences provided they start in corresponding states. Once one has obtained this equivalent state table, the procedure in references [1], [2], or [3] may be applied to minimize the function. The rules for finding this equivalent state table follow. The proof that the tables are indeed equivalent is given in the Appendix. This result is implicit in heuristic discussions by McCluskey [6], Unger [8], and others.

RULES FOR CONSTRUCTING AN EQUIVALENT STATE TABLE THAT TAKES ADVANTAGE OF R1 AND R2

- 1) For each state p in the state table P , assign a state p' in the equivalent state table P' .
- 2) For each column associated with an input i in the state table P , assign a column associated with the same input i in the equivalent state table P' .
- 3) The next state entries in the equivalent state table P' are found by using the relationship:

$$N(p', i) = (S(p, i))'.$$

- 4) The output entries in the equivalent state table P' are found by using the relationship:

$$\phi(p', i) = \phi(S(p, i), i).$$

Example 1

An application of these rules to the state table shown in Fig. 4 results in the equivalent state table shown in Fig. 5. The state minimization procedures when applied to the state table in Fig. 5 show that states C and D in this table are compatible and may be merged together to produce a four-state state table.

Example 2

Under certain conditions, this method may even reduce the number of states in a state table where restrictions R1 and R2 apply and

	I_1	I_2	I_3	I_4
A	$\textcircled{A}, 0$	B, d	D, 0	d, d
B	$\textcircled{B}, 1$	$\textcircled{B}, 1$	C, d	C, 1
C	D, 0	G, d	$\textcircled{C}, 0$	$\textcircled{C}, 1$
D	A, 0	C, d	$\textcircled{D}, 0$	$\textcircled{D}, 1$
G	A, 0	$\textcircled{G}, 0$	C, 0	$\textcircled{G}, 0$

d = don't care

Fig. 4. State table for an asynchronous sequential circuit restricted by R1 and R2.

	I_1	I_2	I_3	I_4
A	$\textcircled{A}, 1$	B, 1	$\textcircled{A}, 0$	$\textcircled{A}, 1$
B	C, 0	$\textcircled{B}, 1$	d, d	A, 1
C	$\textcircled{C}, 0$	B, 0	A, 0	d, d

Fig. 6. Example of an overspecified state table.

every transition is between pairs of stable states. Such a case is shown in Fig. 6. Application of the state minimization procedures to this figure indicates that three states are necessary. On the other hand, application of these procedures to the equivalent state table in Fig. 7 constructed according to the aforementioned rules shows that two states are sufficient. States B and C in this figure can be merged together.

This reduction is due to the overspecification of the state table in Fig. 6 with respect to restriction R2. In particular, the state table should have a B, d entry rather than a B, 0 entry for the present state-input combination of C, I_2 . A second overspecification of a C, 0 entry rather than a C, d entry for the present state-input combinations of B, I_1 has no effect upon the ability to reduce the state table.

Finally, we give a simple corollary, previously proved by Reed [7].

Corollary

If all the outputs of the stable total states in an asynchronous sequential function are specified and if the only don't cares are output don't cares and if R1 and R2 are in effect, then the equivalent (as defined above) state table has no don't cares. Consequently, minimization in this case should be exceptionally simple.

SUMMARY

We have shown that it is fairly easy to alter the present state minimization methods to take account of two common restrictions for an asynchronous sequential function.

APPENDIX

Theorem

The state tables P and P' (formed from P using Rules 1 through 4) are equivalent.

Proof

Let us consider what happens when we apply the input sequence $i_1, i_2, \dots, i_n$ to P , starting in stable state p_0 and progressing through

	I_1	I_2	I_3	I_4
A	$\textcircled{A}, 0$	B, 1	D, 0	d, d
B	$\textcircled{B}, 1$	$\textcircled{B}, 1$	C, 0	C, 1
C	A, 0	G, 0	$\textcircled{C}, 0$	$\textcircled{C}, 1$
D	A, 0	G, 0	$\textcircled{D}, 0$	$\textcircled{D}, 1$
G	A, 0	$\textcircled{G}, 0$	C, 0	$\textcircled{G}, 0$

Fig. 5. State table taking advantage of restrictions R1 and R2; equivalent to the state table in Fig. 4.

	I_1	I_2	I_3	I_4
A	$\textcircled{A}, 1$	B, 1	$\textcircled{A}, 0$	$\textcircled{A}, 1$
B	C, 0	$\textcircled{B}, 1$	d, d	A, 1
C	$\textcircled{C}, 0$	B, 1	A, 0	d, d

Fig. 7. State table taking advantage of restrictions R1 and R2; equivalent to the state table in Fig. 5.

stable states $p_1, p_2, \dots, p_n$. According to restriction R2, we must apply i_1 to p_0 until P is in stable state p_1 , then change i_1 to i_2 with p_1 still the present state, and apply i_2 until P is in stable state p_2 with input i_2 , etc. In general, $p_j = S(p_j, i_j)$ and $p_{j+1} = S(p_j, i_{j+1})$. Thus, we pass successively through the following total states (with the corresponding outputs indicated below):

$$\begin{aligned} & (p_0, i_1), (*_0), (p_1, i_1), (p_1, i_2), (*_1), (p_2, i_2), (p_2, i_3), (*_2), (p_3, i_3) \dots \\ & \phi(p_0, i_1), \phi(*_0), \phi(p_1, i_1), \phi(p_1, i_2), \phi(*_1), \\ & \phi(p_2, i_2), \phi(p_2, i_3), \phi(*_2), \phi(p_3, i_3) \dots \end{aligned}$$

where the general terms are

$$\begin{aligned} & (p_j, i_j), (p_j, i_{j+1}), (*_j), (p_{j+1}, i_{j+1}) \\ & \phi(p_j, i_j), \phi(p_j, i_{j+1}), \phi(*_j), \phi(p_{j+1}, i_{j+1}). \end{aligned}$$

In the positions indicated by the asterisks, there may be unstable total states. The input in these unstable states is the same as on both sides of the asterisk.

The outputs in the unstable states $(p_j, i_{j+1}), (*_j)$ are limited by the situation of the outputs in the two surrounding stable states, (p_j, i_j) and (p_{j+1}, i_{j+1}) . The outputs of these two stable surrounding states may be:

- Case 1: Specified and equal, say $= \phi_*$.
- Case 2: Specified and unequal.
- Case 3: One specified, the other unspecified.
- Case 4: Both unspecified.

In Case 1, the outputs in $(p_j, i_{j+1}), (*_j)$ must all equal ϕ_* or else there would be more than one output change, a violation of R2. Thus, we can eliminate the sequence $\phi(p_j, i_{j+1}), \phi(*_j)$ in this case and still have a compatible output sequence.

In Case 2, in order to satisfy R2 there must not be more than one don't care among $\phi(p_j, i_{j+1}), \phi(*_j)$. Furthermore, all outputs preceding the don't care (or if there is no don't care, then at some point in the sequence) must be equal to $\phi(p_j, i_j)$ while all the outputs after the don't care must equal $\phi(p_{j+1}, i_{j+1})$. In this case, the elimination of $\phi(p_j, i_{j+1}), \phi(*_j)$ also leaves a compatible output sequence.

In Case 3, all the outputs $\phi(p_j, i_{j+1})$, $\phi(*_j)$ must equal the one which is specified in order to satisfy R2. Again, we may eliminate $\phi(p_j, i_{j+1})$, $\phi(*_j)$ and maintain compatibility.

In Case 4, there cannot be any unstable states, $(*_j)$ and (p_j, i_{j+1}) , again by R2. So $\phi(p_j, i_{j+1})$, $\phi(*_j)$ does not exist and may be eliminated and still we have a compatible sequence.

Thus, if input sequence $i_1, i_2, \dots, i_n$ is applied to P with P starting in p_0 , then the output sequence is compatible with

$$\phi(p_1, i_1), \phi(p_2, i_2), \dots, \phi(p_j, i_j), \dots, \phi(p_n, i_n).$$

Now if we apply the same input sequence, $i_1, i_2, \dots, i_n$ to P' (the state table formed from P using Rules 1 to 4) starting in p'_0 , we will pass through states

$$p'_0, p'_1, p'_2, \dots$$

because $N(p'_j, i_{j+1}) = (S(p_j, i_{j+1}))'$ by Rule 3.
 $= p'_{j+1}$ by assumption.

Thus, P' will pass through the following total states (in this case there are no unstable states between the ones shown) with the output sequence indicated below:

$$(p'_0, i_1), (p'_1, i_1), (p'_1, i_2), (p'_2, i_2) \dots$$

$$\phi(p'_0, i_1), \phi(p'_1, i_1), \phi(p'_1, i_2), \phi(p'_2, i_2)$$

where the general terms are

$$(p'_j, i_j), (p'_j, i_{j+1}), (p'_{j+1}, i_{j+1})$$

$$\phi(p'_j, i_j), \phi(p'_j, i_{j+1}), \phi(p'_{j+1}, i_{j+1}).$$

Now since

$$\phi(p'_j, i_j) = \phi(S(p_j, i_j), i_j) \quad \text{by Rule 4}$$

$$= \phi(p_j, i_j) \quad \text{by assumption}$$

and

$$\phi(p'_j, i_{j+1}) = \phi(S(p_j, i_{j+1}), i_{j+1}) \quad \text{by Rule 4}$$

$$= \phi(p_{j+1}, i_{j+1}) \quad \text{by assumption}$$

and

$$\phi(p'_{j+1}, i_{j+1}) = \phi(S(p_{j+1}, i_{j+1}), i_{j+1}) \quad \text{by Rule 4}$$

$$= \phi(p_{j+1}, i_{j+1}) \quad \text{by assumption}$$

then the output sequence with input $i_1, i_2, \dots, i_n$ starting in state p_0 is also

$$\phi(p_1, i_1), \phi(p_2, i_2) \dots \phi(p_j, i_j), \dots, \phi(p_n, i_n).$$

Thus, we have shown that state tables P and P' (formed from P using Rules 1 through 4) have compatible output sequences when any input sequence $i_1, i_2, \dots, i_n$ is applied to both and P starts in p_0 while P' starts in p'_0 . Similarly, it may be shown that the output sequences are compatible if the initial states are p_k and p'_k where p_k is any state in P and p'_k is the corresponding state in P' . Thus, P and P' are equivalent.

REFERENCES

- [1] S. Ginsburg, "A technique for the reduction of a given machine to a minimal-state machine," *IRE Trans. on Electronic Computers*, vol. EC-8, pp. 441-449, September 1959.
- [2] M. C. Paull and S. H. Unger, "Minimizing the number of states in incompletely specified sequential functions," *IRE Trans. on Electronic Computers*, vol. EC-8, pp. 356-367, September 1959.
- [3] A. Grasselli and F. Luccio, "A method for minimizing the number of internal states in completely specified sequential networks," *IEEE Trans. on Electronic Computers*, vol. EC-14, pp. 350-366, June 1965.
- [4] M. C. Paull, "The minimization of the number of states of a completely specified sequential function having restrictions on its allowable input sequences—I," (unpublished).
- [5] E. J. McCluskey, "Fundamental mode and pulse mode operations of sequential circuits," *Information Processing 1962, Proc. IFIP Congress 62*, pp. 725-730.
- [6] —, *Introduction to the Theory of Switching Circuits*. New York: McGraw-Hill, 1965, page 199.
- [7] I. S. Reed, "Some remarks on state reduction of asynchronous circuits by the Paull-Unger method," *IEEE Trans. on Electronic Computers (Short Note)*, vol. EC-14, pp. 262-265, April 1965.
- [8] S. H. Unger, "Hazards and delays in asynchronous sequential switching circuits," *IRE Trans. on Circuit Theory*, vol. CT-6, pp. 12-25, March 1959.

A Modification of Lee's Path Connection Algorithm

SHELDON B. AKERS, JR., SENIOR MEMBER, IEEE

Lee [1] has considered the use of a computer to solve path connection problems such as those encountered in the layout of printed circuit boards. By applying techniques developed for the *shortest route problem* [2]-[4] he has shown that such problems as finding the shortest path between two points or finding the path which crosses the smallest number of other paths may be easily solved. The reader is referred to this paper [1] for a complete discussion of these algorithms and the class of problems which they will solve.

Here we shall examine only the simplest (and yet potentially the most useful) of these algorithms—that used for finding the shortest path between two cells on a rectangular grid containing barriers. A measure of the elegance of Lee's method is the fact that one simple example will suffice to disclose its essential features.

Consider the rectangular grid of Fig. 1. Assume that the shaded cells are *filled* and the unshaded cells are *empty*. Is it possible to find a path through *adjacent* empty cells connecting cell *A* to cell *B*? (Two cells are *adjacent* if they share a common edge.) Moreover, if there are such paths, which one is shortest, i.e., contains the smallest number of cells?

Lee's algorithm proceeds as follows. First a "1" is written in each empty cell adjacent to cell *A* (Fig. 2). Next a "2" is written in each empty cell adjacent to a cell containing a "1" (Fig. 3). Next "3's" are entered in empty cells adjacent to "2's," etc., etc. Eventually one of two things must happen. Either the stage will be reached where cell *B* is still empty and yet no more empty cells can be filled—in which case no path from *A* to *B* exists—or, as is the case here (Fig. 4), a number will be entered into cell *B*.

In this latter case the user not only knows that a path exists from *A* to *B*, but he also knows its optimum length—in this case, 13. Now all that remains is to trace out the path. This is done by starting at *B* and simply proceeding to successively lower numbers until *A* is reached (Fig. 5). Often there may be a choice as to which "next lower" cell to select (see the 2-cell in the path of Fig. 5). In this case any such cell may be selected and the process continued. (With printed circuit boards, for example, a rule for making this selection might be, "Don't change direction when not necessary.")

The purpose of this note is to suggest a modification of the above procedure which can result in a considerable saving in the amount of computer memory necessary for its implementation. As described above, each cell will, at any given step in the process, be in one of a number of different states. It may be empty; it may be filled; or it may contain a 1, 2, 3, ..., or L where L is the maximum length path which can be expected. Thus the number of bits which must be stored for each cell is of the order of $\log(L+2)$. Typical printed circuit boards have a rectangular grid consisting of cells with 50-mil sides. Paths of up to 8 or 10 inches (i.e., 160 to 200 units) are not uncommon.

Hence, typical storage requirements are often in the neighborhood of seven or eight bits per cell. With boards measuring 15 or 20 inches on a side, this can result in a considerable storage problem. The following modification reduces this storage requirement to *two* bits per cell *regardless of the value of L* .

First, it may be noted that in each of Figs. 1 through 5 any cell which contains a number, X , has adjacent to it only cells which are filled or empty or else contain either $(X-1)$ or $(X+1)$. It easily follows from the rules of the procedure that this must always be the case. This fact suggests that instead of using the sequence 1, 2, 3, 4, ... for successive entries into the cells, a simpler sequence (i.e., one involving fewer different symbols) will suffice *provided the predecessor of any symbol in the sequence is different from its successor*.

Manuscript received July 12, 1966; revised October 27, 1966.

The author is with the Electronics Laboratory, General Electric Company, Syracuse, N. Y.